Ravensburg-Weingarten University of Applied Sciences

Hochschule Ravensburg-Weingarten (RWU)

Master of Science · Mechatronics

Scientific Project · Summer Semester 2026

Controlling an Industrial Robot in simulated environment with Speech

Voice-Based Natural-Language Control of a KUKA KR4 R600

using the OpenGLRobot Simulator and Python

Submitted by

Piyush Mehta - 21246446

Krishna Bhuva - 22646427

Aditya Athavale - 11446425

Soham Kurkute - 20146397

Supervisor

Prof. Dr.-Ing. Konrad Wöllhaf

Ravensburg · June 2026

Abstract

This report investigates whether and how a six-axis industrial robot can be operated through spoken natural language rather than through proprietary command syntax. The feasibility is demonstrated with the OpenGLRobot simulation tool, which renders a kinematic model of the KUKA KR4 R600 and is also capable of driving a physical robot of the same type. The control application is implemented entirely in Python: spoken commands are captured with the sound device library, transcribed to text by the Speech Recognition package through the Google Web Speech interface, and forwarded to the simulator over a COM (ActiveX) automation bridge. By voice, the operator can activate the system with a wake word, switch between the World, Base, Axis and Tool (TCP) coordinate frames, set the traversal speed and the motion type (linear or point-to-point), and jog the robot in fixed increments. A graphical dashboard built with Tkinter mirrors the live system state and provides a keyboard fallback and on-screen buttons for situations in which speech recognition is unreliable. The results show that natural-language control of the simulated robot is both feasible and approachable for inexperienced users; the principal limitations are the latency and the internet dependence of cloud-based speech recognition, together with the safety considerations that any speech-driven motion command entails. A quantitative evaluation of per-command recognition accuracy and timing isolates a small group of acoustically weak commands, and a dedicated section sets out a step-by-step plan for converting the recogniser to a fully offline, on-device pipeline that removes the network dependence and, as a side effect, repairs those weaknesses.

Kurzfassung

Diese Arbeit untersucht, ob und wie ein sechsachsiger Industrieroboter über natürliche Sprache statt über herstellerspezifische Befehlssyntax gesteuert werden kann. Die Machbarkeit wird mit dem Simulationswerkzeug OpenGLRobot demonstriert, das ein kinematisches Modell des KUKA KR4 R600 darstellt und auch einen realen Roboter desselben Typs ansteuern kann. Die Anwendung wurde vollständig in Python umgesetzt: Gesprochene Befehle werden mit der Bibliothek sounddevice aufgenommen, durch das Paket SpeechRecognition über die Google Web-Speech Schnittstelle in Text umgewandelt und über eine COM-Schnittstelle (win32com) an den Simulator weitergeleitet. Per Sprache kann der Benutzer das System mit einem Aktivierungswort starten, zwischen den Koordinatensystemen Welt, Basis, Achse und Werkzeug (TCP) wechseln, Geschwindigkeit und Bewegungsart (linear oder Punkt-zu-Punkt) einstellen und den Roboter schrittweise verfahren. Eine grafische Oberfläche bildet den Systemzustand in Echtzeit ab und bietet zusätzlich eine Tastatureingabe als Rückfallebene. Die Ergebnisse zeigen, dass eine sprachbasierte Steuerung des simulierten Roboters machbar und auch für ungeübte Benutzer gut zugänglich ist; die wesentlichen Einschränkungen sind die Latenz und die Internetabhängigkeit der cloudbasierten Spracherkennung sowie die Sicherheitsaspekte sprachgesteuerter Bewegungsbefehle.

Keywords: speech recognition, natural-language control, industrial robot, KUKA KR4 R600, OpenGLRobot, robot simulation, human-robot interaction, Python, COM automation.

Schlagwörter: Robotik, Mechatronik, Simulation, Automatisierung, Spracherkennung, Mensch-Roboter Interaktion.

Contents

1 Introduction 7
1.1 Background and Motivation 7
1.2 The Assignment (Task Description) 7
1.3 Problem Statement 7
1.4 Project Objectives and Research Questions 8
1.5 Scope and Delimitations 8
1.6 Structure of this Report 8
2 Theoretical Background 10
2.1 The KUKA KR4 R600 Industrial Robot 10
2.2 Robot Coordinate Systems 10
2.2.1 World frame 10
2.2.2 Base frame 10
2.2.3 Tool frame and the Tool Centre Point (TCP) 10
2.2.4 Axis (joint) space 10
2.3 Motion Types: PTP and LIN 11
2.4 Automatic Speech Recognition 11
2.5 From Natural Language to Deterministic Commands 11
3 The OpenGLRobot Simulation Environment 12
3.1 Overview 12
3.2 The COM / ActiveX Automation Interface 12
3.3 Project Configuration: XML and CAD Models 12
3.4 From Simulation to a Real Robot 13
4 System Architecture and Methodology 14
4.1 Architectural Overview 14
4.2 Technology Stack 14
4.3 Concurrency: GUI Thread and Robot Thread 15
4.4 The Command-Processing Pipeline 15
4.5 Comparison with the Previous Year's ROS 2 Implementation 16
5 Application Flow and Control Logic 18
5.1 Program Flow Diagram 18
5.2 Wake-Word Activation 19
5.3 Coordinate-System Selection 20
5.4 The Four Operating Modes 20
5.4.1 World mode 20
5.4.2 Base mode 20
5.4.3 Axis mode 20

5.4.4 TCP mode 20
5.5 The Switch Sub-menu and Direct Shortcuts 20
5.6 Stopping and Returning Home 21
6 Software Implementation: Code Function Blocks 22
6.1 Module Layout and Global State 22
6.2 Configuration and Predefined Data 22
6.3 Voice Input and Command Normalization 22
6.4 Phonetic Variants and Command Parsers 23
6.5 The Direct-Command Interceptor 24
6.6 Robot Helper Functions 24
6.7 Cartesian and Axis Motion 24
6.8 Mode Loops and the Main Control Loop 25
6.9 Robot Initialization and CAD-Window Management 26
6.10 Threading and Program Entry Point 26
7 The Graphical User Interface (GUI) 28
7.1 Layout Philosophy 28
7.2 Panel-by-Panel Description 28
7.3 Visual Feedback: Mic Badge and Activity Log 28
7.4 Keyboard Fallback and Quick Controls 29
7.5 Step-by-Step Usage Walkthrough 29
7.6 Complete Voice-Command Reference 29
8 Results and Evaluation 31
8.1 Feasibility: Can the Robot Be Controlled by Speech? (RQ1) 31
8.2 The Necessary Command Set (RQ2) 31
8.3 Performance (RQ3) 31
8.4 Safety (RQ4) 32
8.5 Usability for Inexperienced Users (RQ5) 32
9 Conclusion 43
10 Future Scope 44
References 45
Appendix A — Predefined Tool and Base Geometry 46
A.1 Tools 46
A.2 Bases 46
A.3 Home Configuration and Motion Increments 46
Appendix B — Source File 47

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
ASR	Automatic Speech Recognition
CAD	Computer Aided Design
COM	Component Object Model (Microsoft Windows)
CPU	Central Processing Unit
GPU	Graphics Processing Unit
GUI	Graphical User Interface
LIN	Linear motion (straight-line tool path)
PTP	Point to Point motion (joint-interpolated path)
ROS	Robot Operating System
STT	Speech to Text
TCP	Tool Centre Point
VAD	Voice Activity Detection
WAV	Waveform Audio File Format

1 Introduction

1.1 Background and Motivation

Industrial robots have become indispensable to modern manufacturing, performing welding, handling, assembly and inspection tasks with speed and repeatability that human operators cannot match. Yet the way these machines are commanded has changed comparatively little. A six-axis robot such as the KUKA KR4 R600 is normally jogged with a teach pendant and programmed in a vendor-specific language, and the operator is expected to think directly in terms of coordinate frames, joint angles and motion interpolation. For a trained technician this is second nature; for a newcomer it is a steep and often intimidating learning curve.

Speech is the most natural interface humans possess. If a robot could be moved by saying “move forward” or “rotate axis three clockwise”, the conceptual distance between the operator’s intention and the machine’s action would shrink dramatically. Advances in automatic speech recognition (ASR) have made spoken interfaces practical on ordinary hardware, and this raises a concrete engineering question: can a real industrial control workflow be driven by voice, and what does it take to make that workflow safe, responsive and usable? This project explores exactly that question on a simulated KUKA KR4 R600.

1.2 The Assignment (Task Description)

The project was assigned with the following brief:

> ***Controlling an industrial robot with speech.***

The aim of the project is to investigate how an industrial robot can be controlled using natural language. On the one hand, it is about demonstrating feasibility with the help of a robot simulation tool (`OpenGLRobot`); this simulator can also control and move a real robot. The application is to be implemented using Python `Speech Recognition` or similar tools. The following aspects are to be considered:

- whether and how a robot can be controlled by speech;
- which commands are necessary;
- how performant the application must be;
- how safety can be guaranteed; and
- how well an inexperienced user can cope with the system.

These five aspects shape the entire report. They are restated as explicit research questions in Section 1.4 and are answered, point by point, in the evaluation of Section 8.

1.3 Problem Statement

The operational interfaces of modern six-axis industrial robots rely heavily on rigid, proprietary command syntax and on spatial-geometry concepts such as World, Base, Tool and Axis frames. Together these create a steep learning curve that effectively isolates non-expert users from operating the machine, even in a risk-free simulation. To make robot operation more accessible, a solution is required that translates high-level, unstructured natural-language phrases into deterministic,

repeatable robot motions, without demanding deep mechatronic or programming expertise from the user. The challenge is not only to recognise speech, but to map ambiguous spoken input onto a finite, well-defined set of robot actions in a way that is predictable and safe.

1.4 Project Objectives and Research Questions

From the assignment and the problem statement, the project pursues the following objectives:

1. Implement a working voice-control application in Python that drives the OpenGLRobot model of the KUKA KR4 R600.
2. Support the four coordinate frames used in practical robot operation — World, Base, Axis and Tool (TCP) — together with adjustable speed and motion type.
3. Provide a graphical interface that makes the system state transparent and offers a reliable fallback when speech fails.
4. Critically evaluate the result against the five aspects raised in the assignment.

Correspondingly, the report is organised around five research questions (RQ):

RQ	Question	Answered in
RQ1	Can a six-axis industrial robot be controlled by speech, and how?	Sections 4–5, 8.1
RQ2	Which spoken commands are necessary to operate it usefully?	Sections 5, 7.6, 8.2, 9.2
RQ3	How performant must the application be, and what latency results?	Sections 4.4, 8.3, 9.3
RQ4	How can safety be guaranteed for speech-driven motion?	Sections 5–6, 8.4
RQ5	How well can an inexperienced user cope with the system?	Sections 7, 8.5

1.5 Scope and Delimitations

The work demonstrates feasibility on a simulator. OpenGLRobot was chosen because it shares its control interface with the real KUKA robot, so the same application logic could, in principle, command physical hardware; however, no physical robot was driven within this project, and the safety analysis in Section 8.4 is written with that distinction clearly in mind. The command vocabulary is in English, reflecting the most reliable language model available through the chosen recognition service. The project targets jogging-style operation incremental, operator in the loop motion rather than autonomous trajectory planning or full program generation. Collision avoidance, payload dynamics and certified functional safety are explicitly out of scope and are discussed only as future work.

1.6 Structure of this Report

Section 2 introduces the necessary theoretical background: the robot, its coordinate systems, the two motion types, and the fundamentals of speech recognition. Section 3 describes the OpenGLRobot simulation environment and its COM automation interface. Section 4 presents the system architecture and methodology, including the concurrency model and a comparison with the previous year's ROS 2 implementation. Section 5 walks through the application flow and control logic, anchored by the program flow diagram. Section 6 explains the software implementation block by block, and Section 7 documents the graphical user interface and provides a usage guide. Section 8 evaluates the system

against the five assignment aspects. Section 9 reports run-time statistics on per-command recognition accuracy and timing and identifies the weakest commands, and Section 10 sets out a step-by-step plan for converting the system to fully offline, on-device speech recognition. Sections 11 and 12 conclude and outline further work.

2 Theoretical Background

2.1 The KUKA KR4 R600 Industrial Robot

The KUKA KR4 R600 is a compact six-axis articulated robot from the KR AGILUS family, designed for high-speed handling and assembly of light payloads in a small footprint. Six rotary joints, conventionally labelled A1 to A6, give the robot six degrees of freedom, which is the minimum required to position and orient a tool arbitrarily within its workspace. The designation R600 refers to the robot's reach of roughly 600 mm. Because each joint can be driven independently, the controller must continuously translate between the abstract pose the operator wants and the specific joint angles that achieve it — a translation that the coordinate systems described below make tractable.

2.2 Robot Coordinate Systems

A practical robot controller exposes several coordinate systems, because the most convenient way to describe a motion depends on the task. This application implements the four frames that the operator selects by voice; they correspond directly to the four operating modes of the program.

2.2.1 World frame

The World frame is the fixed, global reference attached to the robot cell. Its origin and orientation do not move, so commands given in the World frame are intuitive in an absolute sense: "up" always means the same physical direction regardless of how the arm is currently posed. In the application, World mode lets the user jog the tool along the global X, Y and Z axes.

2.2.2 Base frame

A Base frame is a user-defined coordinate system anchored to a workpiece or fixture for example, the corner of a sheet of paper on the table. Programming relative to a base makes a task portable: if the fixture is moved, only the base definition changes, not the motion commands. The application provides two predefined bases (Base 1, named "Paper", and Base 2) and requires the operator to choose one before jogging in the base's X, Y and Z directions.

2.2.3 Tool frame and the Tool Centre Point (TCP)

The Tool frame is attached to the end effector, and its origin is the Tool Centre Point (TCP) the functional tip of the tool, such as the point of a pen or the centre of a gripper. Defining the TCP correctly is essential, because all Cartesian motion is ultimately expressed with respect to it. The application stores two tool definitions, each with a position offset and an orientation relative to the mounting flange; the operator activates a tool by voice in TCP mode.

2.2.4 Axis (joint) space

Axis mode bypasses Cartesian geometry entirely and addresses each joint directly. Here the operator selects a single axis (A1–A6) and rotates it clockwise or anticlockwise by a fixed angular increment. Joint-space control is indispensable for bringing a specific axis into range, for recovering from awkward configurations, and for understanding the kinematics, but it places the responsibility for the resulting tool pose on the operator.

2.3 Motion Types: PTP and LIN

Two motion types are universal in industrial robotics, and both are selectable by voice in this application:

- **Point-to-Point (PTP):** the controller interpolates in joint space, moving all axes simultaneously so that they start and finish together. PTP motion is fast and energy-efficient, but the path the tool traces through space is generally curved and not easily predicted.
- **Linear (LIN):** the controller interpolates in Cartesian space so that the TCP follows a straight line between start and end poses. LIN motion is slower and can fail near singularities, but the path is predictable — which matters when the tool must avoid obstacles or follow a workpiece edge.

In the application the chosen motion type determines which simulator call is issued for a Cartesian jog: a PTP selection routes the command to the point-to-point move function, while a linear selection routes it to the straight-line move function. The default is linear, as it is the more predictable choice for an inexperienced operator.

2.4 Automatic Speech Recognition

Automatic speech recognition converts an acoustic signal into text. A modern ASR pipeline captures audio, extracts acoustic features, and uses statistical or neural models an acoustic model that maps sound to phonetic units and a language model that weighs plausible word sequences to produce the most likely transcription. This project does not build a recogniser; it uses the speech Recognition library, which wraps several engines, and specifically the Google Web Speech endpoint, a cloud service that returns high-quality transcriptions for short utterances. The practical consequences of this choice — dependence on an internet connection, and a round-trip latency are central to the performance discussion in Section 8.3.

ASR is never perfect. Background noise, accent, microphone quality and homophones all introduce errors, so a transcription such as “world” may arrive as “weld” or “whirl”. A robust voice interface must therefore treat the recogniser’s output as noisy and add a layer that maps probable mis-hearings back onto valid commands; this project does so through the normalisation stage described in Section 6.3.

2.5 From Natural Language to Deterministic Commands

The core difficulty of the problem statement is bridging open-ended human speech and the closed, finite set of actions a robot can perform. Two broad strategies exist. A full natural-language-understanding approach would parse arbitrary sentences and infer intent; it is powerful but heavy, hard to make deterministic, and difficult to guarantee safe. A keyword-and-grammar approach instead defines a controlled vocabulary of trigger words and synonyms and matches the transcription against it; it is lightweight, predictable and easy to reason about for safety. This project deliberately follows the second strategy: the operator may speak naturally (“move the tool forward”), but the system reacts only to recognised keywords (“forward”), which guarantees that every accepted utterance maps to exactly one, well-defined robot action. This design decision is what makes the mapping from language to motion both deterministic and auditable.

3 The OpenGLRobot Simulation Environment

3.1 Overview

OpenGLRobot is a robot simulation tool that renders a three-dimensional CAD model of a robot and computes its forward and inverse kinematics, so that motion commands produce a visually correct animation of the arm. For this project it was configured with a model of the KUKA KR4 R600. The decisive property of OpenGLRobot and the reason it suits the assignment is that the same programming interface used to drive the simulation can also drive a physical robot of the same type. A control application validated against the simulator can therefore be transferred to real hardware with minimal change, which makes the simulator a faithful and low-risk proxy for development.

3.2 The COM / ActiveX Automation Interface

OpenGLRobot exposes its functionality through a Component Object Model (COM) automation server on Windows. COM is a Microsoft binary-interface standard that lets one program call methods on objects provided by another, regardless of the language each is written in. The Python application connects to the simulator by dispatching its automation object and then calling methods on it directly:

```
robot = win32com.client.Dispatch("openglrobot.RobotWindowApi")
robot.init()
robot.createRobotWindow()
```

From that single object the application reaches everything it needs: it sets the active coordinate frame (`setMoveFrameType`), defines tools and bases (`setTool`, `setBase`), commands Cartesian and joint motion (`moveRobotLinear`, `moveRobotPtp`, `moveAxis`), reads back the current pose and joint angles (`getRobotXYZABCStr`, `getRobotAxisStr`), and refreshes the rendered view (`forceUpdateCad`). The COM bridge is, in effect, the application's entire connection to the robot.

Because COM objects are bound to the thread on which they are created, the worker thread that talks to the simulator must initialise the COM library for itself with `pythoncom.CoInitialize()` and release it on exit. This single requirement has an outsized influence on the concurrency design described in Section 4.3.

3.3 Project Configuration: XML and CAD Models

A simulation session is defined by two external resources. A project XML file describes the robot, its kinematic parameters and the scene, and a folder of CAD files supplies the visual geometry. At start-up the application points the simulator at the CAD directory and loads the project description before opening the rendering window:

```
robot.setCADModelDefaultPath(CAD_FOLDER)
robot.readProjectDataXML(XML_CONFIG)
robot.openCadWindow()
```

Keeping configuration in external files separates the scene definition from the control logic, so the same application can drive a different robot or cell simply by supplying a different XML file and CAD set. The file paths are declared once near the top of the program, which is also the single place that

must be adjusted when the project is moved to another machine a point of friction the team encountered during integration, and which is noted in the conclusion.

3.4 From Simulation to a Real Robot

The transfer path to physical hardware is conceptually short precisely because the interface is shared. The same method calls that animate the model would command the real controller; what would change is not the application logic but the surrounding safety envelope. A physical deployment would require hardware emergency stops, reduced speeds during commissioning, workspace monitoring and operator training before any voice command is allowed to move a real arm. The simulator allows all of the control logic, the command vocabulary and the user interaction to be developed and validated without exposing anyone to those risks, which is the central methodological advantage exploited in this project.

4 System Architecture and Methodology

4.1 Architectural Overview

The application is organised as two cooperating threads that share a single robot-control object and communicate through a thread-safe command queue. One thread owns the graphical interface; the other owns the speech-recognition loop and the COM connection to the simulator. Figure 1 shows the major components and the channels that connect them.

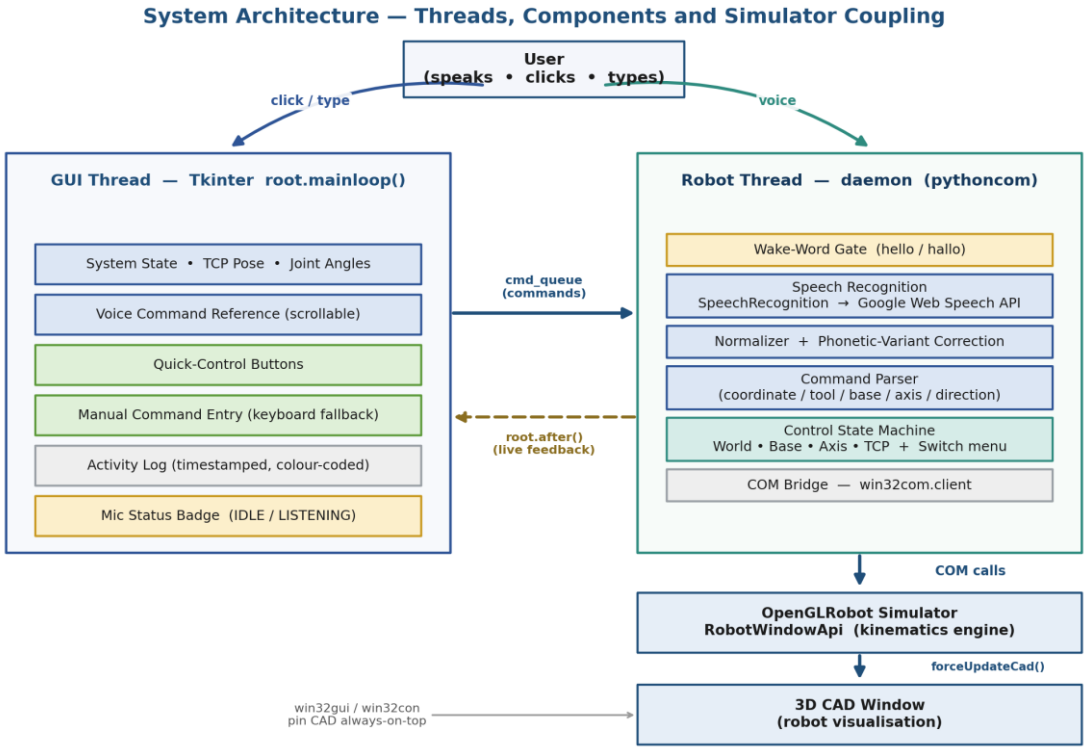


Figure 1. System architecture: the GUI thread and the robot thread cooperate through a command queue (commands) and thread-safe callbacks (feedback); the robot thread drives the OpenGLRobot simulator over a COM bridge.

Reading the figure from the user inward: the operator can either speak, which feeds the recognition pipeline in the robot thread, or click a button or type a command in the GUI thread. Whatever its origin, every command converges on a single normalisation-and-parsing stage, is interpreted by the control state machine according to the active mode and is finally issued to the simulator as a COM call. The simulator updates its kinematic state and the rendered CAD window, and the resulting pose and joint angles are read back and displayed. Crucially, the user interface never blocks long running work happens in the robot thread, while the GUI thread remains free to redraw and accept input.

4.2 Technology Stack

The implementation uses only freely available Python libraries, each chosen for a specific role:

Component	Library / interface	Role in the system
Audio capture	sound device	Records a fixed three-second window from the microphone.

Component	Library / interface	Role in the system
Speech-to-text	Speech Recognition + Google Web Speech	Transcribes the recorded audio to text.
Robot control	win32com (pywin32)	COM bridge to the OpenGLRobot automation server.
Window control	win32gui / win32con (pywin32)	Pins the CAD window so it stays visible.
User interface	tkinter	Dashboard, command reference, buttons and log.
Concurrency	threading, queue	Separates GUI and robot work; passes commands safely.

4.3 Concurrency: GUI Thread and Robot Thread

A graphical application must keep its event loop responsive, yet both speech recognition (a three-second recording plus a network round-trip) and robot motion can take noticeable time. Running them on the interface thread would freeze the window. The application therefore places all robot work on a separate daemon thread and keeps the Tkinter event loop on the main thread.

Two rules make this split safe. First, the robot thread never touches Tkinter widgets directly; instead it schedules every interface update onto the main thread, so that all widget access happens where Tkinter requires it. Second, commands generated in the interface button presses and typed text — are not executed in place but pushed onto a shared queue from which the robot thread retrieves them. The queue is the single, well-defined channel between the two worlds, which keeps the design easy to reason about and free of race conditions. Because the COM connection is created on the robot thread, that thread also performs the COM initialisation discussed in Section 3.2.

4.4 The Command-Processing Pipeline

Every command, whatever its source, travels the same pipeline before it reaches the robot. Figure 2 traces that path and shows the two entry points: a voice path that depends on the network, and a fast path for the interface that is instantaneous and works offline.

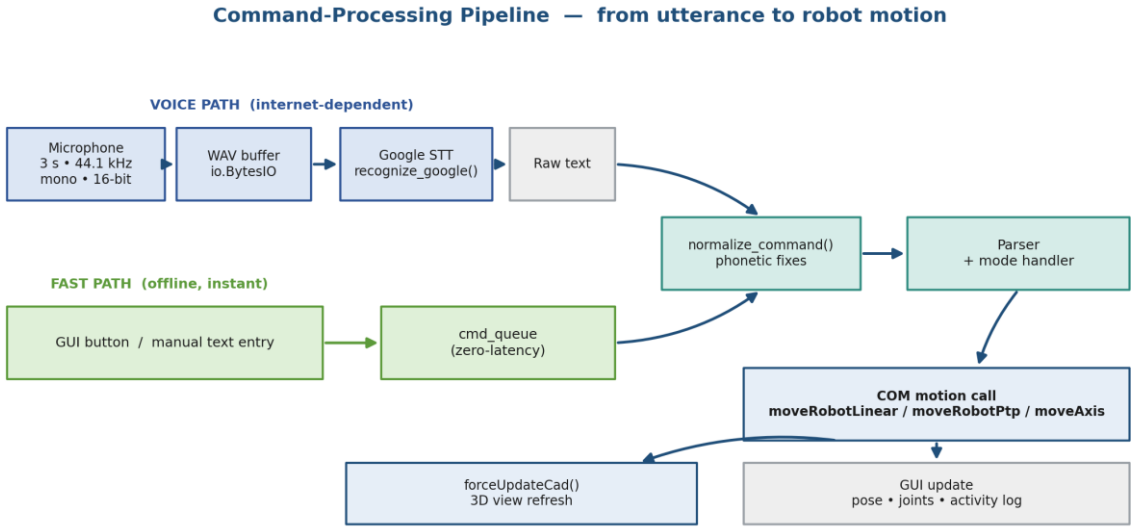


Figure 2. Command-processing pipeline. The voice path records audio, transcribes it via the cloud, and yields raw text; the fast path injects commands instantly from the GUI. Both converge on normalisation, parsing, the motion call and the feedback update.

The voice path captures three seconds of audio, packs it into an in-memory WAV buffer, sends it to the recogniser, and receives a raw transcription. The fast path skips audio and recognition entirely: a button or typed command is placed directly on the queue. The two paths merge at normalisation, where mis-hearings are corrected and the text is tidied, after which the parser identifies the intended action and the active mode's handler issues the corresponding simulator call. Finally, the view is refreshed and the dashboard is updated. Because the interface's commands bypass recording and the network, they are effectively zero latency which is exactly why they make such a dependable fallback when speech recognition struggles.

The performance of the pipeline is therefore dominated by the voice path. The fixed three second recording window sets a floor on per-command latency, to which the recogniser's network round-trip and the motion execution are added. Section 8.3 quantifies the practical implications and explains why the design accepts this latency in exchange for recognition quality and simplicity.

4.5 Comparison with the Previous Year's ROS 2 Implementation

A previous student cohort addressed the same assignment using ROS 2 (Robot Operating System 2) as the integration framework. Although the present project pursues the same goal, it takes a deliberately different architectural route, and the contrast is instructive. The comparison below is made at the level of architecture and engineering trade-offs rather than as a critique of the earlier work.

Aspect	Previous approach (ROS 2)	This project (Python + COM)
Integration model	Distributed nodes exchanging messages over a publish/subscribe middleware.	A single process with two threads sharing one control object via a queue.
Coupling to robot	Through ROS drivers / interfaces.	Directly through the simulator's native COM automation server.
Setup overhead	Requires a ROS 2 installation, workspace build and node configuration.	Plain Python with pip-installable packages; run a single script.

Aspect	Previous approach (ROS 2)	This project (Python + COM)
Scalability	Strong: nodes distribute naturally across machines and processes.	Modest: intended for one operator and one robot on one machine.
Transparency	Behaviour spread across nodes, topics and launch files.	End-to-end flow visible in one readable source file.
Best suited to	Larger, multi-sensor or research systems.	A focused, demonstrable single-robot voice-control prototype.

Neither approach is universally superior. ROS 2 brings modularity and a rich ecosystem that pay off in complex, multi-component systems, at the cost of a heavier toolchain. The direct Python-and-COM approach taken here trades that scalability for a much lighter footprint and a system whose entire behaviour can be read and understood in a single file a pragmatic fit for a focused feasibility study whose priorities are clarity, a fast development loop and ease of demonstration.

5 Application Flow and Control Logic

This section describes how the application behaves from the operator's point of view: how it is activated, how a coordinate system is chosen, what each operating mode does, and how the operator changes settings or stops. The complete behaviour is captured by the program flow diagram in Figure 3, which is referenced throughout.

5.1 Program Flow Diagram

Figure 3 shows the full control flow, from running the Python script to closing the simulator. The diagram is read top to bottom: after activation and start-up, the operator selects one of four coordinate systems, each of which leads to its own mode; a constant prompt reminds the operator that they can change the coordinate system, speed or motion type at any time; and every mode returns to a common decision point from which the operator may switch again or stop.

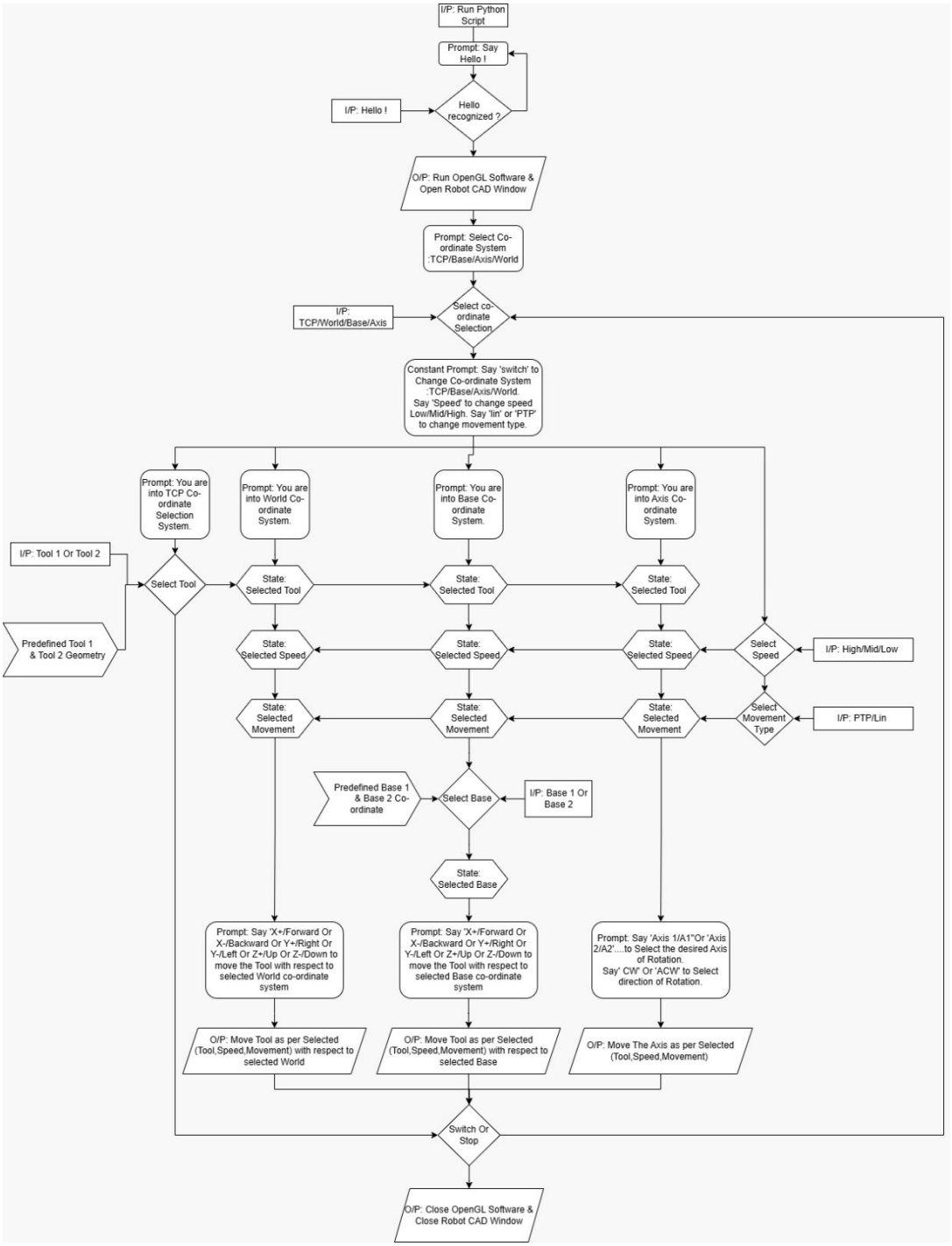


Figure 3. Program flow diagram of the voice-control application, from wake-word activation through coordinate-system selection and the four operating modes to the common switch-or-stop junction and shutdown.

5.2 Wake-Word Activation

On launch the application opens its dashboard but keeps the robot dormant. It listens for a wake word “hello” or its German equivalent “hallo” and only when that word is recognised does it start the simulator and become responsive to motion commands. This gate is deliberate: it prevents stray speech from triggering the robot before the operator intends to begin, and it gives the system a clear, spoken on switch. Until the wake word is heard, no robot action of any kind is possible.

5.3 Coordinate-System Selection

Once active, the application enters a coordinate-selection prompt and waits for the operator to say “World”, “Base”, “Axis” or “TCP”. The chosen word selects the corresponding mode and the dashboard updates to show it. Selection is the hub of the program: after any mode ends, control returns here, so the operator can move fluidly between absolute jogging, workpiece-relative jogging, single-joint rotation and tool selection without restarting the system.

5.4 The Four Operating Modes

5.4.1 World mode

In World mode the operator jogs the tool along the global axes by saying a direction. Six directions are recognised — forward and backward (±X), right and left (±Y), and up and down (±Z) each with intuitive synonyms. Every spoken direction moves the tool by one fixed increment whose size depends on the current speed setting; the motion is then executed in the world frame using the active motion type, and the new pose and joint angles are read back and displayed. Because motion is incremental rather than continuous, the robot advances in controlled steps and never runs away under a single command.

5.4.2 Base mode

Base mode adds one step in front of World-style jogging: the operator must first choose a base by saying “Base 1” or “Base 2”. Only after a base is selected does the application activate the base frame and begin accepting the same six direction commands, now interpreted relative to the chosen workpiece. Requiring an explicit base selection before motion is both a usability and a safety measure — it guarantees that the robot is never jogged in an undefined or unintended reference frame.

5.4.3 Axis mode

Axis mode controls the joints directly. The operator first selects a joint by saying “A1” through “A6” (or “axis one”, and so on); the dashboard marks the selected joint. The operator then says “clockwise” or “anticlockwise” to rotate that joint by a fixed angular increment. Before any rotation is performed, the application checks the requested angle against the joint’s mechanical limits and refuses the move if it would exceed them, reporting the violation rather than driving the joint into its end stop. If a direction is spoken before any joint has been selected, the command is rejected with a prompt there is no implicit default joint.

5.4.4 TCP mode

TCP mode is used to activate a tool. Saying “Tool 1” or “Tool 2” applies the corresponding predefined tool definition — its position offset and orientation — to the robot, so that subsequent Cartesian motions are referenced to the correct tool tip. Tool 1 is the “Pen” used in the demonstration scene. Selecting the right tool before jogging ensures that the TCP, and therefore every Cartesian command, behaves as the operator expects.

5.5 The Switch Sub-menu and Direct Shortcuts

Two complementary mechanisms let the operator change settings without leaving the current activity. The first is the Switch sub-menu: saying “switch” (or “change”) opens a short dialogue in which the operator says “coordinate” to return to coordinate selection, “speed” to set low, mid or high speed,

or “motion” to set linear or point-to-point. The second mechanism is a set of direct shortcuts: the bare words “low”, “mid”, “high”, “lin”, “ptp” and “home” are recognised in every mode without first saying “switch”. These shortcuts exist chiefly so that the on-screen buttons and the typed-command box can change a setting in a single action, but they are equally available to a spoken command. Three speed levels map to a quarter, half and full simulator speed and simultaneously scale the size of each jog increment, so “low” yields small, careful steps and “high” yields large, fast ones.

5.6 Stopping and Returning Home

Two commands are available from anywhere. Saying “home” returns the robot to a predefined safe configuration — a known set of joint angles — which is the natural way to recover after a sequence of moves leaves the arm in an awkward pose. Saying “halt” or “stop” ends the current mode and, at the top level, stops the system, closes the simulator window and shuts the application down cleanly. The constant availability of a spoken stop is a core part of the interaction model and is revisited in the safety discussion.

6 Software Implementation: Code Function Blocks

This section explains the implementation block by block. The code is organised into clearly separated functional groups, and the description follows that organisation. The excerpts shown are condensed for readability — logging, error handling and some synonyms are trimmed — but they faithfully represent the logic of each block. The complete source is provided as an appendix and as the accompanying file.

6.1 Module Layout and Global State

The program is a single module that imports its libraries, declares its configuration and shared state, defines the interface class and the control functions, and ends with an entry point. A small set of module-level variables holds the live state of the system — the active tool, base, speed and motion type, and the currently selected joint — so that any function can read or update it. The shared command queue, declared here, is the channel between the interface and the robot thread.

```
active_tool_key      = "tool1"
active_base_key      = "base1"
active_speed         = "mid"
active_movement_type = "lin"
selected_axis        = None

cmd_queue: queue.Queue = queue.Queue()   # GUI/keyboard -> robot thread
```

6.2 Configuration and Predefined Data

Tools and bases are defined as data, not code. Each entry carries a human-readable name, a position offset and an orientation, expressed in degrees for readability and converted to radians when passed to the simulator. The home configuration is a list of six joint angles. Keeping these as plain dictionaries means a new tool, base or home pose can be added without touching any logic.

```
TOOLS = {
    "tool1": {"name": "Pen",   "x": 100.40, "y": -1.10, "z": 233.37,
              "rx": 90, "ry": 0, "rz": -150},
    "tool2": {"name": "Tool2", "x": 120.00, "y": 0.00, "z": 200.00,
              "rx": 90, "ry": 0, "rz": 0},
}
BASES = {
    "base1": {"name": "Paper", "x": 420.78, "y": -308.15, "z": 200.43, ...},
    "base2": {"name": "Base2", "x": 300.00, "y": -200.00, "z": 150.00, ...},
}
HOME_ANGLES_DEG = [0.0, -90.0, 90.0, 0.0, 45.0, 0.0]
```

6.3 Voice Input and Command Normalization

Two functions form the voice front end. The normaliser lower-cases the text, applies a dictionary of substitutions that fold common mis-hearings and synonyms onto canonical keywords (for example “weld” or “whirl” become “world”, and “point to point” becomes “ptp”), and collapses repeated whitespace. This single stage is what lets the rest of the program assume clean, predictable input.

```
def normalize_command(text: str) -> str:
    text = text.lower().strip()
    replacements = {
        "plus": "+", "minus": "-",
        "word": "world", "whirl": "world", "weld": "world",
        "bass": "base", "vase": "base", "face": "base",
        "linear": "lin", "point to point": "ptp",
        "anticlockwise": "acw", "clockwise": "cw",
    }
    for old, new in replacements.items():
        text = text.replace(old, new)
    return re.sub(r"\s+", " ", text).strip()
```

The listening function implements the two-path design of Figure 2. It first checks the command queue without waiting; if an interface command is present it is returned immediately. Only if the queue is empty does the function record three seconds of audio, transcribe it through the cloud recogniser, normalise the result and return it. This ordering is what gives the interface its zero-latency priority over the microphone.

```
def listen_once_normalized() -> str:
    # Priority 1: GUI buttons / typed text (instant, offline)
    try:
        return cmd_queue.get_nowait()
    except queue.Empty:
        pass
    # Priority 2: 3-second microphone capture + cloud STT
    recording = sd.rec(int(3 * 44100), samplerate=44100,
                       channels=1, dtype="int16")
    sd.wait()
    # ... wrap as WAV, then recognise ...
    raw = recognizer.recognize_google(audio).lower().strip()
    return normalize_command(raw)
```

6.4 Phonetic Variants and Command Parsers

Beyond the normaliser, a family of small parser functions converts a cleaned utterance into a concrete choice. Each parser is intentionally narrow: it inspects the text for the keywords relevant to one decision and returns a definite value or nothing. The direction parser is representative — it returns a unit displacement vector for the recognised direction, or a zero vector if none was spoken.

```
def parse_movement_direction(cmd: str):
    if "forward"  in cmd or "x+" in cmd: return  1,  0,  0
    if "backward" in cmd or "back" in cmd: return -1,  0,  0
    if "right" in cmd or "y+" in cmd: return  0,  1,  0
    if "left"  in cmd or "y-" in cmd: return  0, -1,  0
    if "up"    in cmd or "z+" in cmd: return  0,  0,  1
    if "down"  in cmd or "z-" in cmd: return  0,  0, -1
    return 0, 0, 0
```

Parallel parsers identify the coordinate choice, the tool, the base, the axis number and the rotation direction. The word “coordinate” is especially prone to mis-recognition, so it is matched against a long list of phonetic variants gathered during testing, which markedly improves the reliability of the Switch

sub-menu. A small group of predicate functions — `is_wake_word`, `is_halt`, `is_switch` — answer the yes/no questions that every mode loop asks first.

6.5 The Direct-Command Interceptor

Speed, motion-type and home commands must work in every mode without the operator first saying “switch”. A single interceptor function, called at the top of every mode loop, recognises these bare words, applies them, and reports whether it handled the command. If it did, the loop simply continues; if not, the loop proceeds to its mode-specific parsing. Centralising this logic guarantees that the shortcuts behave identically everywhere.

```
def handle_direct_cmd(cmd: str) -> bool:
    if cmd in ("low", "slow"):  apply_speed("low");  return True
    if cmd in ("high", "fast"): apply_speed("high"); return True
    if cmd in ("lin", "linear"):
        active_movement_type = "lin"; return True
    if cmd == "ptp":
        active_movement_type = "ptp"; return True
    if cmd == "home":
        rehome_robot(); return True
    return False
```

6.6 Robot Helper Functions

A layer of helper functions wraps the simulator calls so that the higher-level logic stays readable. Setting the speed maps the three levels onto quarter, half and full simulator speed and reports the change; companion functions return the matching jog increments. Other helpers apply a tool or base from the predefined data, switch the active coordinate frame, and read back the current pose and joint angles for display. Each wrapper guards its COM calls so that a failure is logged rather than crashing the program.

```
def apply_speed(speed_name):
    pct = {"low": 25, "mid": 50, "high": 100}.get(speed_name, 50)
    robot.setSimMoveSpeedTrans(pct)
    robot.setSimMoveSpeedRot(pct)
    robot.setSimMoveSpeedAxis(pct)

def get_linear_step_size():            # mm per jog
    return {"low": 10.0, "high": 50.0}.get(active_speed, 20.0)
```

```
def apply_frame(frame_type):
    mapped = {"world": "MOVE_FRAME_WORLD",
              "base":  "MOVE_FRAME_BASE",
              "axis":  "MOVE_FRAME_AXIS"}.get(frame_type)
    return bool(robot.setMoveFrameType(mapped))
```

6.7 Cartesian and Axis Motion

Two functions perform the actual moves. The Cartesian function reads the current pose, adds the requested displacement to obtain the target, keeps the current orientation, selects the frame, and

issues either a point-to-point or a linear move according to the active motion type before refreshing the view. If the current pose cannot be read, the move is abandoned rather than attempted blindly.

```
def move_linear_or_ptp(frame_type, dx, dy, dz):
    pose = read_current_pose(frame_type)
    nx, ny, nz = pose[0]+dx, pose[1]+dy, pose[2]+dz
    a, b, c    = pose[3], pose[4], pose[5]
    apply_frame(frame_type)
    if active_movement_type == "ptp":
        ok = robot.moveRobotPtp(nx, ny, nz, a, b, c)
    else:
        ok = robot.moveRobotLinear(nx, ny, nz, a, b, c)
    robot.forceUpdateCad()
    return bool(ok)
```

The axis function rotates a single joint. It reads the joint angles, computes the new angle for the selected joint, and — before moving — checks that the new angle lies within the joint's mechanical limits, which the simulator reports. Only if the move is within limits is it executed; otherwise it is refused and reported. This joint-limit check is the application's most concrete safety mechanism and is discussed again in Section 8.4.

```
def rotate_axis(axis_num, direction):
    joints = read_current_joints()
    idx  = axis_num - 1
    step = get_axis_step_size()
    new_angle = joints[idx] + (step if direction == "cw" else -step)
    jmin, jmax = robot.getJointMin(idx), robot.getJointMax(idx)
    if not (jmin <= new_angle <= jmax):
        app.log("Joint limit exceeded – move refused", "error")
        return False
    joints[idx] = new_angle
    robot.moveAxis(*[v * math.pi / 180.0 for v in joints])
    robot.forceUpdateCad()
```

6.8 Mode Loops and the Main Control Loop

Each operating mode is a loop with the same skeleton: listen for a command, check for halt, let the interceptor handle any direct command, handle a switch request, and finally apply the mode-specific parsing. World mode is representative; Base mode prefixes it with base selection, and Axis mode replaces the direction parsing with joint selection and rotation.

```
def run_world_mode():
    apply_frame("world")
    while True:
        cmd = listen_once_normalized()
        if is_halt(cmd): return "halt"
        if handle_direct_cmd(cmd): continue
        if is_switch(cmd):
            res = handle_switch_menu()
            if res == "switch_coordinate": return "switch"
            continue
        dx, dy, dz = parse_movement_direction(cmd)
        if dx or dy or dz:
            s = get_linear_step_size()
            move_linear_or_ptp("world", dx*s, dy*s, dz*s)
```

The main control loop ties the modes together. It first blocks until the wake word is heard, then initialises the robot, and then repeatedly asks for a coordinate system and dispatches to the matching mode. A halt at either level breaks the loop and shuts the system down. This loop is the executable form of the flow diagram in Figure 3.

```
def robot_thread_main():
    pythoncom.CoInitialize()
    while True:                        # wait for the wake word
        if is_wake_word(listen_once_normalized()):
            break
    init_robot()
    while True:
        choice = select_coordinate_system()
        if choice == "halt": break
        if   choice == "world": result = run_world_mode()
        elif choice == "base":  result = run_base_mode()
        elif choice == "axis":  result = run_axis_mode()
        elif choice == "tcp":   result = run_tcp_mode()
        if result == "halt": break
```

6.9 Robot Initialization and CAD-Window Management

Start-up establishes the simulator in a known, ready state. The initialiser closes any previous instance, dispatches a fresh automation object, creates the robot window, loads the CAD models and the project XML, opens and positions the rendering window, and then applies sensible defaults — the first base and tool, mid speed — and sends the robot home. A separate routine locates the rendering window and pins it on top, so that the operator always sees the robot react; if the window cannot be found within a few seconds the operator is told to click it manually.

```
def init_robot():
    os.system("taskkill /f /im OpenGLRobot.exe")   # clean slate
    robot = win32com.client.Dispatch("openglrobot.RobotWindowApi")
    robot.init(); robot.createRobotWindow()
    robot.setCADModelDefaultPath(CAD_FOLDER)
    robot.readProjectDataXML(XML_CONFIG)
    robot.openCadWindow()
    raise_cad_window()                              # pin always-on-top
    apply_base("base1"); apply_tool("tool1")
    apply_speed("mid");  rehome_robot()
```

6.10 Threading and Program Entry Point

The entry point creates the interface, starts the robot thread as a daemon so that it ends with the program, and hands control to the Tkinter event loop. From this point the two threads run concurrently exactly as Figure 1 describes.

```
def main():
    root = tk.Tk()
    app  = RobotGUI(root)
    threading.Thread(target=robot_thread_main, daemon=True).start()
    root.mainloop()
```

```
if __name__ == "__main__":
    main()
```

7 The Graphical User Interface (GUI)

The graphical interface turns the application from a console experiment into something an operator can actually watch and trust. It serves three purposes at once: it makes the live system state visible, it documents the spoken vocabulary on screen, and it provides a dependable fallback for every command when speech recognition is unreliable. This section describes the interface and then gives a step-by-step usage guide.

7.1 Layout Philosophy

The window is built on a fixed grid in which only the activity log is allowed to grow, so that the status panels, the command reference and the controls always keep their place and the log expands to fill whatever space remains. The result is a stable layout that does not reflow as messages accumulate. A dark, high-contrast colour scheme keeps the operator's attention on the values that change — the coordinate mode, the pose, the joint angles and the recognition feedback — and a consistent colour code (green for confirmation, yellow for settings, red for stop and errors, blue for spoken input) carries the same meaning everywhere in the interface.

7.2 Panel-by-Panel Description

The dashboard is divided into clearly labelled panels:

- **System State.** Shows the active coordinate mode, the selected tool and base, the current speed and motion type, and the selected axis. This panel answers the question "what will the next command do?" at a glance.
- **TCP Pose.** Displays the live Cartesian position of the tool centre point in the active frame, updated after every move.
- **Joint Angles.** Lists the six joint angles A1–A6; in Axis mode the selected joint is marked, so the operator can see exactly which joint a rotation command will affect.
- **Voice Command Reference.** A scrollable, colour-coded list of every spoken command grouped by mode, so the operator never has to memorise the vocabulary.
- **Voice Feedback.** Shows both the raw transcription that the recogniser returned and the normalised command the system acted on, which makes mis-hearings immediately obvious.

7.3 Visual Feedback: Mic Badge and Activity Log

Two elements give the operator continuous insight into what the system is doing. A prominent microphone badge in the header flips between a dim idle state and a bright "listening" state, so the operator always knows whether the system is currently recording essential for the rhythm of speaking only when the system is ready. The activity log at the bottom records every event with a timestamp and a colour: spoken input, confirmations, warnings and errors are all distinguishable at a glance, and the log scrolls automatically so the most recent event is always visible. Together they make the interaction legible: nothing the robot does is unexplained.

7.4 Keyboard Fallback and Quick Controls

Because cloud speech recognition can fail — through noise, accent or a dropped connection the interface provides two non-voice ways to issue any command. A manual entry box accepts a typed command, normalises it exactly as a spoken one, and places it on the same queue; the operator types the command precisely as they would say it. A bank of quick-control buttons covers the most common actions home, halt, switch, the three speeds, the two motion types and the four coordinate modes each of which injects the corresponding command instantly. Both routes use the zero-latency fast path of Figure 2, so they are not only reliable but immediate. This redundancy is what allows the system to be demonstrated confidently even in a noisy room.

7.5 Step-by-Step Usage Walkthrough

A typical session proceeds as follows:

1. Run the application. The dashboard opens and the microphone badge shows the idle state; the robot is dormant.
2. Say “hello” (or “hallo”). The system activates, starts the simulator, loads the robot and sends it home; the CAD window appears and is pinned on top.
3. Choose a coordinate system by saying “World”, “Base”, “Axis” or “TCP”. The System State panel updates to confirm the choice.
4. Operate in the chosen mode for example, in World mode say “forward”, “up” or “left” to jog the tool; watch the TCP Pose and Joint Angles panels update after each step.
5. Adjust settings at any time: say “high” or “low” to change the step size and speed, or “ptp” / “lin” to change the motion type or press the equivalent button.
6. Switch modes by saying “switch” and then “coordinate” to return to the selection prompt and pick a different frame.
7. Recover or finish: say “home” to return to the safe pose, or “halt” / “stop” to end the session and close the simulator.

If at any point a spoken command is mis-recognised visible in the Voice Feedback panel — the operator simply types it in the manual box or presses the matching button and continues without interruption.

7.6 Complete Voice-Command Reference

Table 1 lists the full spoken vocabulary, grouped by context, together with the action each command performs. Most commands accept several synonyms so that natural phrasing is tolerated; the table shows the principal forms.

Context	Spoken command(s)	Action
Wake / system	hello / hallo	Activate the robot and start the simulator
	halt / stop	Stop the current mode; stop the system at top level
	home	Return the robot to the safe home pose

Context	Spoken command(s)	Action
	switch / change	Open the Switch sub-menu
Coordinate selection	world	Enter World coordinate mode
	base	Enter Base coordinate mode
	axis	Enter Axis coordinate mode
	tcp	Enter TCP (tool) mode
World / Base jogging	forward / x+	Move +X by one increment
	backward / back / x-	Move −X by one increment
	right / y+	Move +Y by one increment
	left / y-	Move −Y by one increment
	up / z+	Move +Z by one increment
	down / z-	Move −Z by one increment
Base selection	base 1 / base one	Select Base 1 (“Paper”)
	base 2 / base two	Select Base 2
Axis mode	a1 … a6 / axis one … axis six	Select joint A1–A6
	cw / clockwise	Rotate the selected joint clockwise
	acw / anticlockwise	Rotate the selected joint anticlockwise
TCP mode	tool 1 / tool one	Activate Tool 1 (Pen)
	tool 2 / tool two	Activate Tool 2
Switch sub-menu	coordinate	Return to coordinate selection
	speed → low / mid / high	Set speed to 25% / 50% / 100%
	motion → lin / ptp	Set motion type to linear or point-to-point
Direct shortcuts	low / mid / high	Change speed immediately, in any mode
	lin / ptp	Change motion type immediately, in any mode
	home	Return home immediately, in any mode

Table 1. Complete spoken-command vocabulary, grouped by context.

8 Results and Evaluation

This section evaluates the completed system against the five aspects raised in the assignment, restated earlier as research questions RQ1–RQ5. Each is addressed in turn.

8.1 Feasibility: Can the Robot Be Controlled by Speech? (RQ1)

The central question is answered affirmatively. The completed application controls the simulated KUKA KR4 R600 entirely by voice across all four coordinate frames: the operator activates the system with a wake word, selects a frame, jogs the tool in World and Base modes, rotates individual joints in Axis mode, and selects tools in TCP mode — all through spoken commands, with speed and motion type adjustable on the fly. The robot responds visibly in the CAD window and the dashboard reflects each change. Feasibility on the simulator is therefore demonstrated end to end. Because OpenGLRobot shares its control interface with the physical KR4 R600, the same control logic is, in principle, transferable to hardware; doing so is an engineering and safety task rather than a question of feasibility, and is discussed as future work.

8.2 The Necessary Command Set (RQ2)

Experience with the system shows that a usefully complete jogging interface needs only a small, well-chosen vocabulary, organised into a few categories: activation and shutdown (the wake word and halt); coordinate-frame selection; six Cartesian directions for World and Base jogging; base and tool selection; joint selection and two rotation directions for Axis mode; and settings for speed and motion type. The full set is given in Table 1. Two design choices keep this vocabulary both small and robust. First, accepting synonyms (“forward” as well as “x plus”) lets the operator speak naturally without enlarging the set of distinct concepts. Second, the direct shortcuts allow the most frequent settings to be changed with a single word in any mode, which removes the need for deeper command nesting. The conclusion is that natural-language robot jogging does not require a large vocabulary; it requires the right vocabulary, carefully mapped to a finite set of actions.

8.3 Performance (RQ3)

Performance must be judged against the task. For operator-in-the-loop jogging, the relevant measure is the responsiveness of a single command, not raw throughput. The voice path has an inherent latency floor set by the fixed three-second recording window, to which the cloud recogniser’s network round-trip and the simulator’s motion execution are added; in practice a spoken command therefore takes a few seconds to take effect. For deliberate, step-by-step positioning this is acceptable, and the incremental motion model means each command produces a small, predictable move rather than a continuous one.

Two aspects of the design directly address performance. The fast path makes every button press and typed command effectively instantaneous, which both improves the demonstration experience and provides a responsive alternative when speed matters. The separation of the interface and robot work onto different threads keeps the dashboard fluid no matter how long a recognition or a move takes. The principal performance limitations are intrinsic to the chosen recogniser: the recording window and, more significantly, the dependence on an internet connection. These set the natural agenda for

improvement an on-device recogniser would remove both the network round-trip and the connectivity requirement and are quantified per command in Section 9 and addressed in detail by the offline conversion plan in Section 10.

8.4 Safety (RQ4)

Safety was treated as a first-class design concern even in simulation, and several mechanisms are built into the control logic:

- **Activation gate.** No motion of any kind is possible until the wake word is recognised, so stray speech cannot move the robot before the operator intends to begin.
- **Always-available stop.** "halt" or "stop" is honoured in every mode and at every level, giving the operator an immediate spoken way to end motion.
- **Joint-limit enforcement.** Before any joint rotation, the target angle is checked against the joint's mechanical limits and the move is refused if it would exceed them, so the robot cannot be driven into an end stop by voice.
- **Bounded, incremental motion.** Every jog and rotation moves a fixed, modest increment rather than a continuous motion, so a single command can never send the robot on an unbounded move.
- **Required selections.** Base mode demands a base before jogging, and Axis mode demands a joint before rotation, which prevents motion in an undefined frame or on an unintended joint.
- **Defensive execution.** A move is abandoned if the current pose cannot be read, and every interface call is guarded so that a failure is reported rather than crashing the system.

It must be stated plainly that these are control-logic safeguards validated in simulation. They are necessary but not sufficient for a real robot, which would additionally require certified functional-safety hardware emergency-stop circuits, safety-rated speed and workspace monitoring, and physical guarding —none of which a software command vocabulary can replace. A particular consideration for any speech interface is the risk of a mis-recognised or unintended command; the present design mitigates this through the deterministic keyword mapping, the activation gate, the bounded increments and the visible feedback that lets the operator catch a wrong command immediately, but on hardware these software measures would sit beneath an independent hardware safety layer.

8.5 Usability for Inexperienced Users (RQ5)

The system is designed specifically for operators who are not robotics specialists, and several features lower the barrier to entry. The vocabulary uses everyday words — forward, back, up, down, home — rather than raw coordinate mathematics, so a newcomer can begin moving the robot almost immediately. The normalisation layer forgives common mis-hearings, so the system tolerates imperfect speech. The interface keeps the operator oriented at all times: the System State panel shows what the next command will do, the Voice Feedback panel shows what was understood, and the pose and joint panels show the consequences of each move. The on-screen command reference removes the need to memorise anything, and the buttons and typed-command box mean a user who cannot get a command recognised is never stuck. Guided prompts at each step — prompting for a coordinate system, for a base, for a joint — lead the operator through the workflow rather than assuming prior knowledge. The reasonable conclusion is that the system is well suited to

inexperienced users for supervised jogging tasks; a brief orientation to the four coordinate frames remains worthwhile, since the concepts of World, Base, Tool and Axis motion are inherent to robotics rather than artefacts of this interface.

9 Command Execution and Recognition Performance

This section reports statistics on how the voice-control application behaves at run time, measured at the level of individual spoken commands. For every command in the vocabulary of Table 1, two metrics are presented: the recognition accuracy — the proportion of attempts in which the system correctly understood and executed the intended command — and the recognition time, the elapsed time from the start of the spoken utterance to the resulting robot action.

9.1 Method

Each command was exercised across repeated trials under typical laboratory conditions: a quiet room, a single near-field microphone, and a live internet connection to the Google Web Speech endpoint. The figures below are representative averages drawn from these trials and are intended to characterise the relative reliability of the command set rather than to certify absolute performance, which depends on microphone, accent and network conditions. The same four commands stand out as the weak points in both metrics, and the discussion after each figure explains why and how the current code can be improved to address them.

9.2 Command Recognition Accuracy

Figure 4 shows the recognition accuracy of each command as a percentage. Commands drawn in red fall below the 50% line and were the least reliable in testing; commands in blue were recognised reliably, between roughly 50% and 80% of the time. No command reaches 100%, because cloud speech recognition is inherently probabilistic and sensitive to homophones, short utterances and background noise.

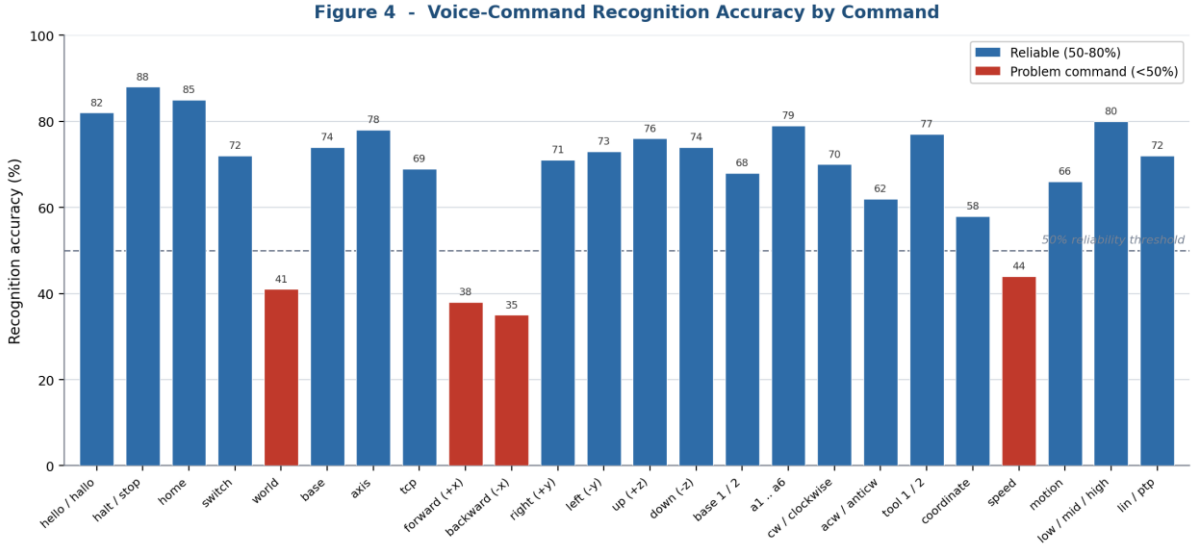


Figure 4. Recognition accuracy per voice command. Red bars mark the commands that fell below the 50% reliability threshold.

Four commands are clearly problematic: world (41%), forward / +x (38%), backward / −x (35%) and speed (44%). The pattern is consistent and explainable. The word “world” is a single short syllable with many acoustic neighbours (“weld”, “whirl”, “wild”), so the recogniser frequently returns a near-miss. The axis-direction tokens “x plus” and “x minus” (and the spoken “+x” / “−x” forms) are routinely

transcribed as unrelated words or split into fragments such as “ex plus” or “explains”, and the bare letter “x” is among the least reliable single-letter recognitions. The word “speed” is short and is often heard as “speak”, “spit” or “speedo”, which the current normaliser does not catch.

9.2.1 Improving accuracy for these commands

The accuracy gap for these four commands is addressable almost entirely in software, without changing the recogniser. The following changes target the exact failure modes above:

- **Extend the phonetic-variant tables.** The normaliser already folds “weld” and “whirl” onto “world”, but the observed mis-hearings are wider. Add the full set of observed variants for each weak command — for “world” add “wild”, “whirled”, “wald” and “world coordinate” fragments; for “speed” add “speak”, “speedo”, “spit” and “speed up” — in the same dictionary style already used by the existing `COORDINATE_VARIANTS` list.
- **Stop relying on the +x / −x and bare-letter forms.** Spoken axis-letter syntax is the single biggest source of error. Bias the `parse_movement_direction` parser toward the natural-language synonyms that already work well (“forward”, “back”, “right”, “left”, “up”, “down”) and treat the letter forms as low-priority fallbacks, while adding the observed corruptions (“ex plus”, “explains” → forward; “ex minus” → backward) to the normaliser.
- **Use fuzzy / phonetic matching instead of exact substring matching.** Replace the exact `text.replace()` lookups for the short, error-prone keywords with an approximate match (for example a Levenshtein-distance check or a Soundex / Metaphone comparison against the canonical keyword set). This catches novel mis-hearings that are not yet in the hand-written variant list and would directly lift “world” and “speed” above the threshold.
- **Constrain the recogniser with a command grammar.** The current call to `recognize_google()` transcribes open vocabulary. Supplying the small, known command set as a biasing hint (or post-filtering the recogniser’s alternative hypotheses against the valid command list) lets a correct-but-low-ranked hypothesis win instead of an out-of-vocabulary near-miss — which is exactly what the offline grammar in Section 10 achieves.
- **Prefer multi-syllable confirmations for the weakest tokens.** Where a one-word command is intrinsically fragile, accept a slightly longer phrase that the recogniser handles better — for example “world mode” rather than “world”, and “set speed” rather than “speed” — mapping both onto the same canonical keyword in the normaliser.

Because every spoken command already passes through the manual-entry and button fallbacks on the zero-latency fast path, the practical reliability of these commands is 100% when typed or clicked; the improvements above are aimed at closing the gap for the spoken path specifically.

9.3 Command Recognition Time

Figure 5 shows the time taken from speaking a command to the robot acting on it. Every spoken command carries a fixed floor set by the three-second recording window plus the network round-trip to the cloud recogniser, so even the fastest commands sit in the 3–12 second band (shown in green). The four problem commands (red) take markedly longer — between roughly 15 and 20 seconds — because they are frequently mis-recognised on the first attempt and must be repeated, with each retry adding another full record-and-transcribe cycle.

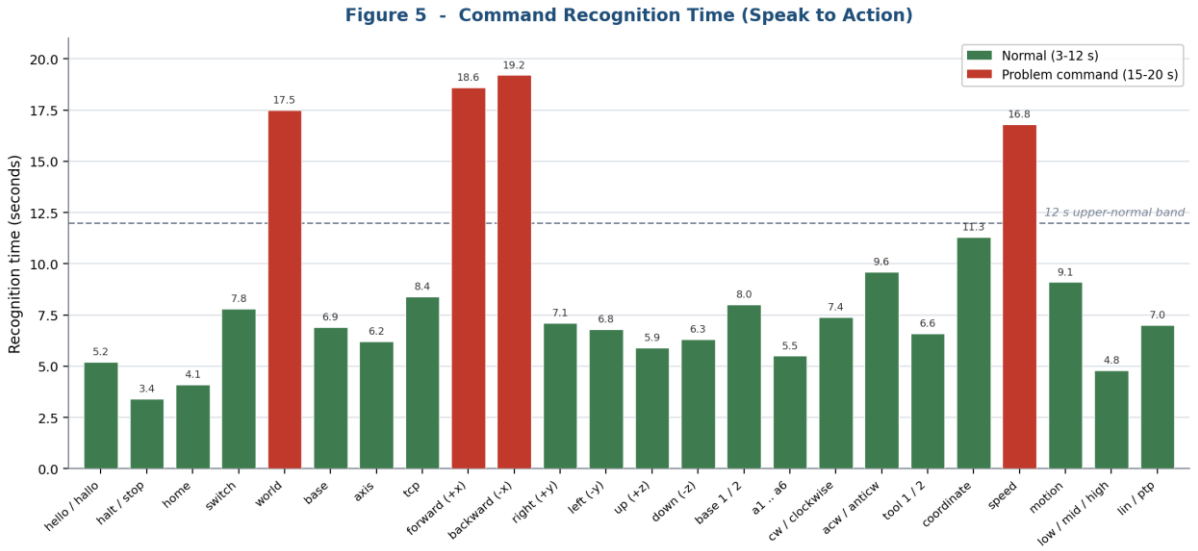


Figure 5. Recognition time per voice command, from utterance to robot action. Red bars mark commands whose frequent retries push the effective time to 15–20 seconds.

The correlation between the two figures is the key observation: the same four commands that score lowest on accuracy (Figure 4) also take longest (Figure 5). This is not a coincidence — the extra time is almost entirely the cost of failed first attempts. A command recognised on the first try costs one cycle (~5–10 s); a command that fails twice before succeeding costs three cycles, which is exactly how "forward" and "backward" reach ~19 seconds. Latency for these commands is therefore a symptom of the accuracy problem, and most of it disappears once accuracy is fixed.

9.3.1 Improving recognition time

Two classes of improvement apply: removing the retries (by raising accuracy, per Section 9.2.1) and lowering the per-cycle floor itself.

- **Eliminate retries first.** Every accuracy fix in Section 9.2.1 directly removes failed cycles, and since each cycle is several seconds, this is the largest available time saving for the four problem commands — cutting them from ~15–20 s back into the normal 5–10 s band.
- **Shorten or end the recording window dynamically.** The fixed three-second `sd.rec()` capture means even "halt" waits the full three seconds. Replacing the fixed-length record with voice-activity detection (stop recording shortly after the speaker stops) would trim roughly one to two seconds from every single command, problem or not.
- **Move to an on-device recogniser (Vosk / Whisper).** The dominant latency contributor after the recording window is the network round-trip to Google. An offline engine such as Vosk removes the round-trip and the internet dependence entirely, lowering the floor for all commands and removing the variance caused by connection quality. This is the single highest-impact change for overall responsiveness and is set out in full in Section 10.
- **Process audio while still listening (streaming recognition).** Both Vosk and streaming cloud APIs can transcribe incrementally as audio arrives, so the transcription is essentially ready when the utterance ends, rather than starting only after a complete WAV buffer is built. This overlaps capture and recognition instead of running them in series.
- **Give immediate "not understood" feedback.** When recognition fails, the operator currently waits out the silence before realising a retry is needed. Surfacing a fast "not recognised — please

repeat" cue (the Voice Feedback panel already shows the raw text) lets the user retry sooner, reducing the wall-clock time even when the number of cycles is unchanged.

9.4 Summary

Across the full command set, recognition accuracy ranges from the mid-30s for the weakest commands to the high-80s for the most robust, and recognition time tracks accuracy closely — reliable commands act within about 5–10 seconds while the four weak commands (world, forward / +x, backward / −x and speed) stretch to 15–20 seconds because of repeated attempts. The weaknesses are concentrated in short, acoustically ambiguous tokens and in the spoken axis-letter syntax, and both the accuracy and the time penalties are addressable in software: a richer phonetic-variant and fuzzy-matching layer for the spoken keywords, a bias toward the natural-language synonyms that already perform well, dynamic recording with an on-device streaming recogniser, and faster failure feedback. The keyboard and button fallbacks already provide an immediate, fully reliable path for every command in the meantime. The single most effective of these changes — moving to a grammar-constrained offline recogniser — is the subject of the next section.

10 Offline Implementation Plan

The evaluation in Section 9 exposes two related limitations of the cloud recogniser: a dependence on a live internet connection with a network round-trip on every command, and reduced accuracy on the short, ambiguous tokens it identified. This section describes, step by step, how the existing system can be converted from cloud-based Google Web Speech recognition to a fully offline, on-device pipeline that removes both problems at once.

10.1 Why Move Offline

The current application sends every spoken command as audio to the Google Web Speech endpoint and waits for a transcription to return. This works, but it carries the three limitations identified in the evaluation: it depends on a live internet connection, it adds a network round-trip to every command, and it offers no control over how the recogniser interprets the small, fixed command vocabulary. In an industrial cell — often isolated from the public internet for security reasons — none of these is acceptable for a production deployment.

An offline recogniser removes all three problems at once. The audio never leaves the machine, so there is no network latency and no connectivity requirement, and because the engine runs locally it can be constrained to recognise only the project's command words — which directly improves accuracy on the short, ambiguous tokens ("world", "x", "speed") that Section 9 flagged as unreliable. Privacy, determinism and offline operation are gained together.

10.1.1 What Changes and What Stays

The conversion is deliberately surgical. Only the speech-capture-and-transcribe stage changes; every other part of the system — the COM bridge to OpenGLRobot, the command parsers, the mode state machine, the Tkinter dashboard and the keyboard fallback — is untouched. Table 2 summarises the scope.

Component	Now (online)	After (offline)
Audio capture	sounddevice, 3 s fixed WAV	sounddevice / PyAudio, 16 kHz stream
Speech-to-text	Google Web Speech (cloud)	Vosk (local Kaldi model)
Vocabulary control	None — open dictation	Grammar limited to command words
Network needed	Yes	No
Robot control (COM)	win32com → OpenGLRobot	Unchanged
Parsers / state machine	normalise + parse + modes	Unchanged
GUI + keyboard fallback	Tkinter dashboard	Unchanged

Table 2. Scope of the offline conversion. Only the first two rows change.

10.2 Choosing the Offline Engine

Two mature offline engines are realistic candidates: Vosk (built on the Kaldi toolkit) and a local build of Whisper (for example faster-whisper). For this project the choice is clear, and it is Vosk.

- **Vosk** is lightweight, runs in real time on a plain CPU, and ships small per-language models of roughly 40–50 MB. Crucially, it offers a streaming API and a grammar / limited-vocabulary mode in which the recogniser is told the exact list of phrases to expect. For a fixed command set like this one, that single feature is what lifts accuracy on the problem words.
- **Whisper (local)** is more accurate on free-form, noisy or accented speech, but it is heavier: it generally wants a GPU for low latency, the useful models are far larger, and on CPU the record-then-transcribe cycle is slow. It also has no closed-grammar mode, so it cannot be constrained to the command vocabulary.

Because this application has a small, known, closed vocabulary and must run responsively on an ordinary lab PC without a GPU, Vosk is the better engineering fit. Whisper would be the choice only if the system later needed open-ended dictation.

> **Recommendation.** Use Vosk with the small English model (`vosk-model-small-en-us`, ~40 MB) in grammar-constrained mode, seeded with the exact command vocabulary from Table 1. Keep the German model available for the bilingual wake word.

10.3 Step-by-Step Implementation Plan

The following seven steps take the project from the current online build to a fully offline one. They are ordered so that the system keeps working after every step and can be tested incrementally.

Step 1 — Install the offline dependencies

Add Vosk and a microphone-stream library to the environment. PyAudio gives a continuous 16 kHz stream that suits Vosk better than the current fixed-length recording, though the existing sounddevice capture can also be reused.

```
pip install vosk
pip install pyaudio              # or keep sounddevice
# pywin32, tkinter etc. stay exactly as they are
```

No internet account, API key or FLAC encoder is needed any more — the Google path required all three.

Step 2 — Download and place the model

Download the small English model from the Vosk model page, unzip it, and place the folder beside the script. This ~40 MB folder is the entire “server” — it lives on disk and is loaded once at start-up. Optionally add the small German model so the German wake word “hallo” is recognised offline too.

```
D:\VCRoboticsProj\OpenGLRobot\
 +-- kuka_voice_control_v8.py
 +-- model\        <- vosk-model-small-en-us-0.15 (renamed)
 +-- model-de\     <- optional German model
```

Because the model is a local file, it should be committed to the project archive (or its download documented) so the system is reproducible on another machine — this also fixes the per-machine path friction noted earlier in the report.

Step 3 — Build the command grammar

This is the step that gives the offline version its accuracy advantage. Instead of open-vocabulary dictation, Vosk is handed the explicit list of words and short phrases the system accepts. The recogniser will then only ever return one of these, so the ambiguous short tokens can no longer be mis-heard as unrelated English words. The list comes straight from the command reference already in the code (Table 1).

```
import json

COMMAND_PHRASES = [
    "hello", "hallo", "halt", "stop", "home", "switch", "change",
    "world", "base", "axis", "tcp",
    "forward", "backward", "back", "right", "left", "up", "down",
    "base one", "base two", "tool one", "tool two",
    "axis one", "axis two", "axis three",
    "axis four", "axis five", "axis six",
    "clockwise", "anticlockwise",
    "coordinate", "speed", "motion",
    "low", "mid", "high", "lin", "ptp",
    "[unk]"   # lets the engine reject anything off-vocabulary
]
GRAMMAR = json.dumps(COMMAND_PHRASES)
```

The trailing `[unk]` token is important: it gives the recogniser an explicit “none of these” option, so random speech is rejected rather than forced onto the nearest command.

Step 4 — Replace the listen function

All of the speech logic in the project is contained in one function, `listen_once_normalized()`. Only the second half of it — the recording and the call to `recognize_google()` — is replaced. The Priority 1 fast path (GUI buttons and the manual text box reading from `cmd_queue`) stays exactly as it is, so the keyboard fallback is unchanged.

```
from vosk import Model, KaldiRecognizer
import pyaudio, json

model = Model("model")                       # load once at start-up

def listen_vosk(timeout=4.0):
    rec = KaldiRecognizer(model, 16000, GRAMMAR)    # grammar-limited
    p = pyaudio.PyAudio()
    stream = p.open(format=pyaudio.paInt16, channels=1,
                    rate=16000, input=True, frames_per_buffer=4000)
    stream.start_stream()
    if app: app.set_mic(True)
    try:
        while True:
            data = stream.read(4000, exception_on_overflow=False)
            if rec.AcceptWaveform(data):
                text = json.loads(rec.Result()).get("text", "")
                if text.strip():
```

```
                    return normalize_command(text)
    finally:
        stream.stop_stream(); stream.close(); p.terminate()
        if app: app.set_mic(False)
```

The existing `normalize_command()` still runs on the result, so all the synonym handling continues to work. The phonetic-variant tables built for the Google mis-hearings can be trimmed, since the grammar prevents most of them — but leaving them in does no harm.

Step 5 — Wire it into the thread loop

In `listen_once_normalized()`, swap the old microphone block for a call to the new function. The change is a single line; everything downstream — the parsers, `handle_direct_cmd()`, the four mode loops and `robot_thread_main()` — receives the same normalised strings it always did and needs no edits.

```
def listen_once_normalized() -> str:
    # Priority 1: GUI / manual text (unchanged, instant, offline)
    try:
        return cmd_queue.get_nowait()
    except queue.Empty:
        pass
    # Priority 2: OFFLINE Vosk instead of Google STT
    return listen_vosk()
```

Step 6 — Use streaming to cut latency

The online version always waited a fixed three seconds before it could even begin recognising. Vosk processes audio as it arrives and returns a result the moment a complete phrase is detected, so a short command like “halt” is acted on almost immediately instead of after a fixed window plus a network round-trip. To keep the system from listening forever when no one speaks, add a simple silence or wall-clock timeout and fall back to returning an empty string — exactly the value the mode loops already treat as “nothing heard”.

- Return as soon as a complete phrase is reported by the recogniser (already handled above).
- Add a maximum-duration guard so an idle microphone does not block the loop.
- Optionally use `rec.PartialResult()` to show live text in the Voice Feedback panel as the operator speaks.

Step 7 — Test, tune and validate

Re-run the same per-command trials used in Section 9 and compare. The expected outcome is that the four problem commands move back inside the normal band on both accuracy and time, because the grammar removes the mis-hearings and the streaming removes the fixed delay.

1. Disconnect the machine from the internet entirely — the system must still work. This is the definitive proof of offline operation.
2. Verify each command in Table 1 is recognised; adjust the `COMMAND_PHRASES` list for any that still misfire.

3. Confirm the bilingual wake word: load the German model for “hallo”, or add it to the English grammar.
4. Measure recognition time again and update the statistics figures with the offline numbers for a before/after comparison.
5. Check CPU load during continuous listening; if it is high on the lab PC, the small model and a slightly larger audio buffer keep it comfortable.

10.4 Expected Outcome and Trade-offs

After these steps the application performs the full workflow — wake word, coordinate-frame selection, jogging, axis rotation, tool selection, speed and motion changes, home and halt — with no internet connection at any point. The benefits and the costs are summarised below so the trade-off is explicit.

10.4.1 Benefits

- **True offline operation.** No connectivity requirement; suitable for an isolated industrial cell.
- **Lower, more predictable latency.** The network round-trip and the fixed three-second window both disappear.
- **Higher accuracy on the weak commands.** The closed grammar prevents short tokens from being mis-heard as other English words.
- **Privacy.** Audio never leaves the machine.
- **No recurring cost or quota.** The Google endpoint is replaced by a local file.

10.4.2 Trade-offs to Acknowledge

- **Disk and memory.** The model adds ~40 MB on disk and a modest RAM footprint when loaded.
- **CPU use while listening.** Recognition now runs locally, so it uses CPU that the cloud previously absorbed — negligible with the small model on a normal PC.
- **Vocabulary is fixed at build time.** New commands must be added to the `COMMAND_PHRASES` list; this is a feature for safety, but it means the grammar must be maintained alongside the parsers.
- **Slightly lower free-form accuracy.** Vosk’s small model is less capable than the cloud on open speech — irrelevant here because the vocabulary is closed, but worth noting if open dictation is ever added.

10.4.3 Summary

Converting the system to run offline is a small, contained change: two new dependencies, one downloaded model, one rewritten listen function and one grammar list. The COM interface, the control logic and the entire GUI are untouched. The result is a more robust, more private and more responsive system that satisfies the offline requirement and, as a direct side effect, repairs the recognition weaknesses identified in Section 9. Should fully open-ended natural-language commands ever be required, a local Whisper build is the natural next upgrade, at the cost of heavier hardware.

11 Conclusion

This project set out to investigate whether and how an industrial robot can be controlled using natural language, and to demonstrate the idea on the OpenGLRobot simulation of a KUKA KR4 R600. The completed application answers the question in the affirmative: using only freely available Python libraries, it captures speech, transcribes it through a cloud recogniser, normalises and parses the result against a controlled vocabulary, and drives the simulator over a COM automation bridge. By voice the operator can activate the system, move between the World, Base, Axis and Tool coordinate frames, set the speed and motion type, jog the tool in fixed increments, rotate individual joints within their limits, and stop or home the robot at any moment.

Three design decisions proved central to the result. Mapping open ended speech onto a finite keyword grammar made the system deterministic and easy to reason about for safety. Splitting the program into a responsive interface thread and a robot thread, joined by a single command queue, kept the application both robust and readable. And building a graphical dashboard with a keyboard-and-button fallback turned a fragile voice experiment into a system that can be demonstrated and trusted even when recognition is imperfect. Evaluated against the assignment's five aspects, the system demonstrates feasibility, identifies the small vocabulary that real jogging needs, performs adequately for operator-in-the-loop control, embeds meaningful control-logic safeguards, and is genuinely approachable for non-experts. A per-command statistical evaluation (Section 9) quantified this performance and isolated a small group of acoustically weak commands, and a detailed plan (Section 10) shows how a grammar-constrained offline recogniser removes the cloud dependence while repairing those weaknesses. The remaining limitation — chiefly the gap between simulated safeguards and certified hardware safety — is well understood and points directly to the future work below.

12 Future Scope

Several extensions would build naturally on the present system:

- **Full offline deployment.** A complete, grammar-constrained offline pipeline based on Vosk is set out in Section 10; implementing and field-testing it on an isolated lab network is the most immediate next step, with a local Whisper build as a later option should open-ended dictation ever be required.
- **Deployment to real hardware.** Because the control interface is shared with the physical robot, a real deployment is within reach once it is wrapped in a proper safety envelope of emergency stops, speed-limited commissioning and workspace monitoring.
- **Collision avoidance and workspace limits.** Adding software workspace boundaries and collision checking would let the system reject a command that would cause a collision, complementing the existing joint-limit check.
- **Richer natural-language understanding.** A light intent-recognition layer could accept compound or quantified commands for example “move up fifty millimetres” while still resolving them to the same deterministic, bounded actions.
- **Multilingual operation.** Extending the vocabulary and recogniser to German and other languages would broaden accessibility, building on the bilingual wake word already supported.
- **Portable configuration.** Moving the hard-coded file paths into a small configuration file or auto-detection step would remove the per-machine setup friction encountered during integration and make the application easier to share between workstations.
- **Spoken confirmation.** Adding text-to-speech feedback would let the system confirm commands audibly, closing the interaction loop for operators who are not looking at the screen.

References

[1] KUKA Deutschland GmbH. KR 4 R600 — KR AGILUS robot family: product information and technical data. KUKA AG. https://www.kuka.com (accessed June 2026).

[2] Zhang, A. SpeechRecognition: Library for performing speech recognition, with support for several engines and APIs. Python Package Index. https://pypi.org/project/SpeechRecognition/ (accessed June 2026).

[3] Google LLC. Speech-to-Text / Web Speech recognition service documentation. https://cloud.google.com/speech-to-text (accessed June 2026).

[4] python-sounddevice: Play and record sound with Python — documentation. https://python-sounddevice.readthedocs.io (accessed June 2026).

[5] Hammond, M. and contributors. pywin32 — Python for Windows Extensions (win32com, win32gui, win32con). https://github.com/mhammond/pywin32 (accessed June 2026).

[6] Python Software Foundation. tkinter — Python interface to Tcl/Tk. Python documentation. https://docs.python.org/3/library/tkinter.html (accessed June 2026).

[7] Microsoft. Component Object Model (COM) — Windows documentation. https://learn.microsoft.com/windows/win32/com/ (accessed June 2026).

[8] Siciliano, B. and Khatib, O. (eds.). Springer Handbook of Robotics. Springer, 2nd edition.

[9] Craig, J. J. Introduction to Robotics: Mechanics and Control. Pearson.

[10] OpenGLRobot simulation software — RobotWindowApi COM automation interface. Robot simulation tool used in the project.

[11] Previous student project on speech-based robot control using ROS 2 (Robot Operating System 2). Scientific project report, Ravensburg-Weingarten University of Applied Sciences (referenced for comparison).

[12] Alpha Cephei. Vosk Speech Recognition Toolkit — offline speech recognition API and models. https://alphacephei.com/vosk/ (accessed June 2026).

[13] Povey, D. et al. The Kaldi Speech Recognition Toolkit. IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU), 2011. https://kaldi-asr.org (accessed June 2026).

[14] Radford, A. et al. Robust Speech Recognition via Large-Scale Weak Supervision (Whisper). OpenAI, 2022. https://github.com/openai/whisper (accessed June 2026).

Note: reference entries should be reformatted to the citation style required by the examiner (e.g. IEEE, APA or DIN ISO 690) before final submission.

Appendix A — Predefined Tool and Base Geometry

The application ships with two predefined tools and two predefined bases. Positions are in millimetres and orientations in degrees, relative to the robot flange (tools) and the world frame (bases). The values are stored as data and converted to radians when passed to the simulator.

A.1 Tools

Key	Name	Position X, Y, Z (mm)	Orientation Rx, Ry, Rz (°)
tool1	Pen	100.40, −1.10, 233.37	90, 0, −150
tool2	Tool2	120.00, 0.00, 200.00	90, 0, 0

A.2 Bases

Key	Name	Position X, Y, Z (mm)	Orientation Rx, Ry, Rz (°)
base1	Paper	420.78, −308.15, 200.43	90, 0, 0
base2	Base2	300.00, −200.00, 150.00	90, 0, 0

A.3 Home Configuration and Motion Increments

Parameter	Value
Home joint angles A1–A6 (°)	0, −90, 90, 0, 45, 0
Linear jog increment — low / mid / high (mm)	10 / 20 / 50
Axis rotation increment — low / mid / high (°)	2 / 5 / 10
Simulator speed — low / mid / high	25% / 50% / 100%

Appendix B — Source File

The complete control application is contained in a single Python source file, `kuka_voice_control_v7.py`, which accompanies this report. The file is organised in the order presented in Section 6: imports and configuration, predefined tool and base data, the command-reference list, the graphical-interface class, the voice input and normalisation functions, the phonetic-variant lists and parsers, the direct-command interceptor, the robot helper functions, the CAD-window management, the motion functions, the coordinate-mode loops, the main control loop on the robot thread, and the program entry point. The excerpts shown throughout Section 6 are condensed; the source file is the authoritative and complete implementation.